

Real-Time Systems - Laboratory Exercise 2 (week 3)

Lecturer: Dr. Pham Viet Hung

Lab Note: These tasks are to be conducted individually (but you should discuss with your friend, don't get lost alone). You should make a **note document** and keep your solution/s **code (*.c file/s)**, so that you can review later when doing the final project, as well as prepare for the In-class Tests.

In the Note document, you should have the required diagrams, screen captures and text to answer the questions. Your note should also contain examples for different input and expected output to demonstrate the behaviour of your program meets the outlined objective. Finally, keep the source files (*.c files) together with the note.

1. Creating a POSIX thread

Here is a simple example of creating a new thread. (see [thread_ex1.c](#))

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

/* This is the code for the thread. It is not called directly
 * but passed as a parameter to pthread_create in the main thread.
 * The single void parameter may be used to pass parameters.
 */
void *thread_ex (void *data)
{
    printf("Thread started\nNow sleeping...\n");
    sleep(20);
    printf("Finished sleeping\nThread finished\n");
    return 0;
}

int main(int argc, char *argv[])
{
    pthread_t th1;
    void *retval;

    // Create and start the thread
    pthread_create (&th1, NULL, thread_ex, NULL);

    /* Suspend the main program (itself a thread)
     * until the thread has terminated.
     */
    pthread_join (th1, &retval);

    printf("Main terminated\n");
    return EXIT_SUCCESS;
}
```

The parameters to pthread_create() are (in order):

th1 Thread identifier, used in other functions such as pthread_join

NULL	May be used to change the initial attributes of the thread Passing NULL means that the thread starts with the default priority (10) and default scheduling strategy (round-robin)
thread_ex	start_function i.e. the function which is run as a thread
NULL	May be used to pass parameters to the thread This may be the actual parameter if there is only one To pass more than one parameter this must be the address of a structure (see thread_ex2.c for an example of passing parameters)

Note: Normally the thread start_function (i.e. the function which is run as a thread) will loop, and only terminate under defined conditions (such as when some error has occurred). Creating multiple threads with the same start_function is allowed. When the main program terminates, any spawned threads that are still running will also be terminated.

TASK 1A: Create a new project in Momentics running the above code. Then add a third thread that waits for the user to type a number before it terminates and call it within the main process – use pthread_join() to ensure the main thread does not terminate while the child thread is waiting for user input. Test the modified program and confirm that it works as expected – you should have the main program (itself a thread) as well as 2 other threads. You should use getpid(), gettid() to display the process ID (pid) and thread (tid) in the console windows running the threads. You should also launch a second console (via telnet session) and run the 'pidin' command (at the # prompt) to list the running processes on the QNX target so that you can match the process ID and thread ID to the Momentics console information.

Answer the following questions in your report:

What are the thread states between 0.1 and 19.99 seconds into execution?

Main thread _____
First child thread _____
Second child thread _____

In your report you should also provide screen capture/s of the (Telnet) console with the pidin command showing the state of the processes/threads during the 0.1 and 19.99 second execution phase for all three threads. You should also upload your *.c file for this solution along with your report.

Changing Thread Attributes

The main thread attributes (given in the pthread_attr_init() description in the Library Reference) are:

Attribute	Default Value	Notes
<i>detachstate</i>	PTHREAD_CREATE_JOINABLE	Alternative value is PTHREAD_CREATE_DETACHED which if used means thread cannot be joined with
<i>inheritsched</i>	PTHREAD_INHERIT_SCHED	i.e. inherit from parent thread. Alternative value is PTHREAD_EXPLICIT_SCHED
<i>schedpolicy</i>	Inherited from parent thread	Actual values include SCHED_FIFO SCHED_RR (round robin) SCHED_OTHER
<i>schedparam</i>	Inherited from parent thread	Parameters include the thread priority
<i>contentionscope</i>	PTHREAD_SCOPE_SYSTEM	This is the recommended

		setting for real-time systems (only allowed value for QNX)
<i>stacksize</i>	4K bytes	May need to increase this for threads with lots of private data and deep nesting of function calls
<i>stackaddr</i>	NULL	i.e. let system set stack location –it is recommended that you do NOT set your own stack location

To access these attributes you use the `pthread_attr_getattribute` functions where *attribute* is the attribute name in the above table. Similarly, to change *attribute* use the `pthread_attr_setattribute` function.

Try looking up each of these functions in the online QNX 7 documentation at:

<http://www.qnx.com/developers/docs/7.0.0/>

You may not need to change the attributes for any of your threads from the default values but in case you do here is an example of explicitly setting thread attributes before the thread is created (not a complete program).

```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>

// define thread function(s) here

int main (void)
{
    pthread_t th1;
    void *retval;
    pthread_attr_t th1_attr;
    struct sched_param th1_param;

    // Initialise thread attribute object to the default values (required)
    pthread_attr_init (&th1_attr);
    // Explicitly set the scheduling policy to round robin
    pthread_attr_setschedpolicy (&th1_attr, SCHED_RR);
    // Set thread priority (can be from 1..63 in QNX, default is 10)
    th1_param.sched_priority = 15;
    pthread_attr_setschedparam (&th1_attr, &th1_param);
    // Now set attribute to use the explicit scheduling settings
    pthread_attr_setinheritsched (&th1_attr, PTHREAD_EXPLICIT_SCHED);
    // Increase the thread stacksize
    pthread_attr_setstacksize (&th1_attr, 8000);

    // Create and start the thread using the explicit attributes
    pthread_create (&th1, &th1_attr, thread_ex, NULL);

    ... //code missing
```

TASK 1B: Combine the above code sample with the code you developed in Task 1A so that the first child thread is scheduled with a higher priority (i.e. 15) than the main thread. Satisfy yourself that the threads are scheduled as expected.

Similar to passing parameters to a function (either by value or reference), you may wish pass parameters to a newly launched thread. Threads have their own special way of handling this. As defined above, the `pthread_create()` function takes 4 argument, 3 of which we have used so far. The last argument can be used to pass the address of a single datatype, or more appropriately a struct (when you have more than one data member you wish to pass).

TASK 1C: Download the [thread_ex2.c](#) file from Canvas and set it up within a QNX Momentics project. Compile and run the program and make sure you understand how it works. You should experiment with changing the values in the struct, which will change how the program behaves.

TASK 1D

Based on the supplied example code and the code you have developed so far, create a program with two threads, one with the default priority of 10 (i.e. the `main()` thread) and one with a priority of 1. The main thread should print characters 'a' to 'j', one at a time, and the other thread (Print_Numbers thread) should print numbers from 0 to 9, one numerical character at a time. Each thread should repeat printing 100 characters before terminating with a newline character printed after every 10 printed characters.

If only the main thread was to run the output should look like this:

```
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
```

You should use the exact same code loop in each thread with the only difference being whether the loop prints numeric or alphabetic characters. Once you have the main threading doing what you want, you can add the Print_Numbers thread to your code – Make sure you spawn the Print_Numbers thread before the loop (within the `main()` function) that prints the alphabetical characters in the main thread.

In your report put a screen capture of the 'pidin' command running in a separate telnet console which shows the priority and scheduling strategy for each thread.

Test the program and satisfy yourself that the threads are scheduled as expected and that the output is as you expect. If everything has gone to plan and you are running the code on a single CPU node (ie. A single CPU VM or a Beaglebone Target) your output should look something like below (mostly characters will be printed first):

```
abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
```

abcdefghij
abcdefghij
abcdefghij
abcdefghij
abcdefghij
0123456789
0123456789
0123456789
0123456789
0123456789
0123456789
0123456789
0123456789
0123456789
0123456789
0123456789

Answer the following questions in your report:

Considering the concept of time-slicing, what do you need to do to get the Print_Numbers thread to print some numbers before the main thread finishes printing its 100 alphabetical characters? _____

Up until now you should have not used any blocking processes (i.e. sleep() or usleep()). What happens when you add a usleep(1) in the main thread just after spawning the print numbers thread and before the loop that prints the 100 alphabetical characters? _____
Why? _____

You can answer the above questions with text, code extracts and/or screen dumps.

2. Processes and Message Passing

In QNX, creating multiple processes is easy – you simply run multiple programs (usually in separate terminal windows or console tabs in Momentics).

It is possible for a process to spawn (run) other programs as an independent process. QNX also has mechanisms that allow you to run programs (processes) on other computers running QNX on the same network.

In real-time systems a process often wants to send or receive data from another process. One way of doing this is using POSIX mqueues (message queues).

Note: Some installations of QNX (i.e. if you make a new target from QNX Software Center) will not have the POSIX mqqueue service running. You can start the service by typing mqqueue at the QNX prompt if the binary file (mqqueue) is located in the /sbin directory. Alternatively, section 13 of the [Getting_Started_with_QNX_at_RMIT.PDF](#) describes how to automate the process of starting mqueues in a freshly installed x86 QNX Target.

Here is an example of a program that sends messages using an mqqueue. A more comprehensive version is available on Canvas: [mqsend.c](#)

```
#include <stdio.h>
#include <mqqueue.h>
#include <sys/stat.h>

#define MESSAGE_SIZE 1000
```

```

#define Q_FLAGS O_RDWR | O_CREAT | O_EXCL
#define Q_Mode S_IRUSR | S_IWUSR

int main(int argc, char *argv[])
{
    printf("Welcome to the QNX Momentics mqueue send process\n");

    mqd_t      qd;
    int      i = 0;
    char buf[MESSAGESIZE] = {};

    struct mq_attr attr; // all members must be initialized
    attr.mq_maxmsg = 100; //queue size – maximum number of messages
    attr.mq_msgsize = MESSAGESIZE; //Maximum size of a message
    attr.mq_flags = 0;
    attr.mq_curmsgs = 0;
    attr.mq_sendwait = 0;
    attr.mq_recvwait = 0;

    struct mq_attr * my_attr = &attr; // must use a pointer to address attributes

    const char * MqueueLocation = "/test_queue";
    qd = mq_open(MqueueLocation, Q_FLAGS, Q_Mode, my_attr);
    if (qd != -1)
    {
        for (i=1; i <= 5; ++i)
        {
            sprintf(buf, "message %d", i);
            printf("queue: '%s'\n", buf);
            mq_send(qd, buf, MESSAGESIZE, 0);
            sleep(2);
        }
        mq_send(qd, "done", 5, 0); // send last message so the receive process knows

        printf("\nWait here for 10 seconds before closing mqueue\n");
        sleep(10);
        mq_close(qd);
        mq_unlink(MqueueLocation);
    }
    else
        printf("\nmqueue could not be opened\n");

    return 0;
}

```

Notes:

See the documentation or help on each of the mq_ functions for a full description of them and their parameters.

In mq_open(), the third and fourth parameters (the mode or queue permissions and the queue attributes respectively) are only required when creating a new mqueue as here, specified by including O_CREAT in the flags (second parameter).

The mode or file permission bits are defined in <sys/stat.h>. In this case the owner is given read and write permission, and others have read permission.

Message queues are listed in the file system namespace in /dev/mqueue. Active queues can be listed e.g. using 'ls -il /dev/mqueue' which will show all active queues and the number of messages waiting in each queue. Note that /dev/mqueue only exists while there are active queues (ie. Before mq_unlink() is called).

You can create multiple message queues on one computer by using different names (change test_queue to any desired name).

Here is an example of a program to receive messages send by the above program. A more comprehensive version is available on Canvas: [mqreceive.c](#)

```
#include <stdio.h>
#include <mqueue.h>

#define MESSAGE_SIZE 1000

int main (void)
{
    mqd_t      qd;
    char buf[MESSAGE_SIZE] = {};

    struct mq_attr attr;

    if ((qd = mq_open("/test_queue", O_RDONLY)) != -1)
    {
        mq_getattr(qd, &attr);
        printf("max. %u msgs, %u bytes; waiting: %u\n",
               attr.mq_maxmsg, attr.mq_msgsize, attr.mq_curmsgs);
        while (mq_receive(qd, buf, MESSAGE_SIZE, NULL) > 0)
        {
            printf("dequeue: '%s'\n", buf);
            if (!strcmp(buf, "done"))
                break;
        }
        mq_close(qd);
    }
    else
        printf("\nmqueue could not be opened\n");

    return 0;
}
```

Task 2A: Compile and run the supplied code in separate QNX C projects within the same Momentics workspace. Then configure the "Run Configurations.." launch macro for each so that you can run the processes on same QNX Target. First adjust the launch macro and run the mqsend process. Wait for it to send 2 or 3 messages before running the mqreceive process. You could also try using Putty or Tera Term to telnet into the QNX target and run the processes in separate other windows. The mqreceive task will quickly remove all waiting messages from the queue then wait for the remaining messages to be sent. If you want the mqueue to stay active for a while, try putting in some user input code in the mqsend so a key stroke is required before it terminates (or use an extended sleep time of 10 or more seconds)

Note: make sure you navigate the /dev/mqueue/test_queue via a Telnet console to confirm to yourself that the queue is made when the send process starts and deleted after the send process terminates.

3. Message Passing between PCs

Normally in QNX, POSIX mqueues are used to pass messages between processes running on the same node or between different processes running on different QNX nodes on the network. To use mqueues between nodes the path name for the message queue must use the full path name. That is the /test_queue is really located at /dev/mqueue/test_queue). The full name of the node (i.e. network path name) also needs to be added for a mqueue created on another PC running QNX.

On a given network all machines running QNX can see each other if the qnet service is loaded into memory. If you don't have a /net directory in your target's file system then you can load qnet by typing the following command at the QNX terminal prompt:

```
# mount -T io-pkt lsm-qnet.so
```

Note: As previously covered, the hostname and MAC address for each QNX node on the network needs to be unique. Check each by using the following commands (only the hostname can be changed):

```
# hostname <optional :NewHostName>
# ifconfig
```

If all is well, the file systems of each node will be accessible via the /net directory on each node. Try the "ls" command at the prompt: `# ls /net`

Note: Having qnet enabled is obviously a potential security hazard, so a QNX network should always be run behind a firewall, or preferably on its own isolated network. If you cannot see /net (e.g. on your own installation of QNX) that means that QNET is not running or there are two QNX targets on the network with the MAC address or hostname. This could happen if a x86 Target VM has been copied and both the original and copy are running on the same network – this will result in an error as they will have the same MAC address and hostname).

If you are running your own x86 QNX VM targets the full pathname to an mqueue named test_queue on another machine (using the hostname of the other machine) could be something like: /net/hostname1/test_queue

It really depends on how the router or DCP server on the network is setup. You can check by running the "ls" command within the /net directory on a node.

There are many potential errors relating to qnet. You may need to revisit the Getting Start with QNX at RMIT guide and ensure you have qnet run properly.

Task 3A: Create your own version of the mqreceive and mqsend code files using the full pathname for the test_queue by replacing the MqueueLocation string with the correct net/ addresses as shown when you navigate the /net directory. Run mqsend on a QNX Target, and then run the modified mqreceive process on another QNX Target before mqsend terminates within a telnet session. You should see that the sent messages are received on the other machine (QNX node).

References

Multithreaded programming with pthreads / Bil Lewis, Daniel J. Berg.

Pthreads programming / Bradford Nichols, Dick Buttlar, and Jacqueline Proulx Farrell

HINT: The code in the provided example programs does not check the return value from most of the function calls in order to show only the essential code. The return value is usually set to -1 if there was an error, and should be checked when safety and reliability is important.